Soup training monitor

Frontend Engineer take-home | 10 November 2026

Repository | Live monitor | Evidence and trace files

Verdict: SHIP for the supplied monitoring scope

The monitor preserves the evidence needed to distinguish unstable training from broken telemetry. Automated truthfulness checks and targeted browser recovery checks pass. Performance measurements are attached, including an unfavorable hover result. This verdict covers the synthetic-data assignment; it does not claim validation of a real training-data ingestion service.

Change first: investigate pointer queueing and inspector updates, with benchmark report serialization moved outside the pointer hot path. Optimized hover p95 is 55.8 ms, above the proposed 50 ms budget. The two slowest samples spend about 30-34 ms queued before the handler. Keep exact source inspection and remeasure before changing the numerical algorithm. This small latency miss does not invalidate the displayed evidence.

What was built, and what the chart preserves

Next.js, TypeScript and Apache ECharts Canvas. Twenty deterministic runs include NaN/Infinity, identical and conflicting duplicates, unsorted steps, known-cadence gaps, zero/negative values, and a million-row run with loss 12 at step 500,000. The UI provides search/filter/sort, separate metric plots, exact summaries, shared ranges, source inspection, quality details, responsive layout, keyboard controls and visible recovery states.

A worker owns the full source. Each metric and continuous segment selects first/minimum/maximum/last observations per visible bucket, retaining original values and chronological order. Zoom/resize recompute from source. NaN, infinity and gaps break the affected line; conflicts expose finite extremes instead of selecting an arbitrary winner. Summaries, domains and inspection use source observations. Linear axes, straight lines and disabled automatic sampling avoid concealed transformations. Bucket extrema and the isolated spike survive; **not every smaller local peak survives**.

Before and after, measured in production

Same million-row fixture and Canvas adapter: raw main-thread rendering versus worker/reduced rendering. Original Chrome Performance traces contain verified **4x CPU throttling**, screenshots and event marks. Times use correlated Paint events; they are not just animation-frame estimates.

Measure	Raw	Worker + reduction
Cold complete chart, median of 5	13,630 ms	864 ms
Cold chart range	8,906-14,874 ms	748-1,215 ms
Zoom p95, 30 commands	909.2 ms	84.0 ms
Mouse-move inspection p95, 30 accepted events	100.6 ms	55.8 ms
Loaded tab footprint / own empty baseline	1.4 GB / 48.3 MB	103 MB / 44.5 MB
Approximate incremental tab footprint	1.35 GB	58.5 MB

M2 MacBook Air, 8 GB RAM, macOS 26.5; running Chrome UA 154.0.0.0; Node 24.13.1. Timing: localhost, cache disabled, DPR 2; matched cold series at 727 x 837 CSS px and matched interactions at 915 x 837. Memory: fresh renderer per variant, DevTools closed, normal full window, at least 30 seconds idle. Values preserve Chrome's rounded display units. The worker shares its tab's renderer footprint and is counted once.

Evidence, failures and limits

598 tests in 19 files passed, plus lint, strict TypeScript, all 20 exact catalog summaries and production build. Source revision: `0eb0e3d`. Full output is in `evidence/performance/check.stdout` and `check.stderr`. The prior independent raw-source audit covers 60 cases, 300 viewports, 900 metric results and 3,891 inspection groups/pages; all fixture fingerprints remained unchanged.

Tests cover the spike across containing/boundary viewports, invalid-value breaks, per-metric duplicate handling, sorted provenance, exact domains/coordinate mappings, column inspection, signed zero and extreme finite values. Independent BigInt checks guard integer bucket membership. Browser records cover the visible spike, narrow ranges, keyboard/mobile recovery, stale run/range/inspection replies, retained snapshot labels after refresh failure, timeout, malformed replies, renderer retry and zero-size recovery. Nine locale/time-zone cases are Node checks. Hosted checks verify routing/reload, assets, worker loading, the spike and keyboard 404 recovery.

The four original compressed traces, raw ledgers, memory screenshots and reproducible CSV are in `evidence/performance/`. Run `python3 scripts/analyze-browser-evidence.py` to correlate all 192 completed samples. Historical failures remain in their evidence folders. The first raw trace also retains an interrupted setup navigation; it is excluded from successful-load statistics and described explicitly.

Load/zoom endpoints conservatively use the last same-frame Paint before completion; hover uses the first Paint after the inspector's DOM commit. The visible first plot is loss; the other charts are below the fold. Mouse movement came from short native drags, with the button held, through the actual mouse-move handler. The shell ignores busy moves (23 raw, 27 optimized); this is a discrete-response test, not a continuous-sweep stress test. Its inspector/event wiring is simpler than the full dashboard. Diagnostic overhead is included. Worker activity is traced, but a 4x slowdown of every worker was not independently calibrated; DevTools throttling is not a machine-wide speed limit.

The optimized zoom maximum is still 171 ms; p95 does not imply every response is under 100 ms. FCP medians are 124 ms raw and 132 ms optimized, so shell paint did not improve. Sampled memory peaks were 1.4 GB and 109 MB. Raw footprint remained 1.4 GB 30 seconds after unload without forced GC. Peak sampling and one release cycle cannot establish leak freedom. JS heap is recorded separately and is not presented as tab memory. Numerical tests do not prove every painted pixel, browser race, screen-reader journey, device or real-world telemetry contract.

Reflection

An unexpected failure was that the reducer and its oracle agreed on a wrong bucket boundary: both reused rounded floating-point arithmetic. Exact quotient/remainder boundaries plus an independent BigInt oracle made that check meaningful. The remaining concern is the gap between correct numbers and a trustworthy experience: stale state, readable labels, input queueing and visible recovery need browser evidence of their own. This reflection describes the recorded implementation; it does not invent independent candidate observations.

AI use and ownership

Codex assisted with planning, documentation, fixture generation, implementation, tests, debugging, browser measurements and this note. Accepted approaches include worker-owned source data and per-metric extrema-preserving reduction. The initial latest-duplicate-wins idea was replaced by explicit conflicts; bucket arithmetic and its oracle were corrected. The candidate directed ECharts, the separate plots, black/cream/orange palette, supplied icon and removal of decorative copy. Automated checks and browser observations were executed by the agent. Independent human code verification is not claimed here.